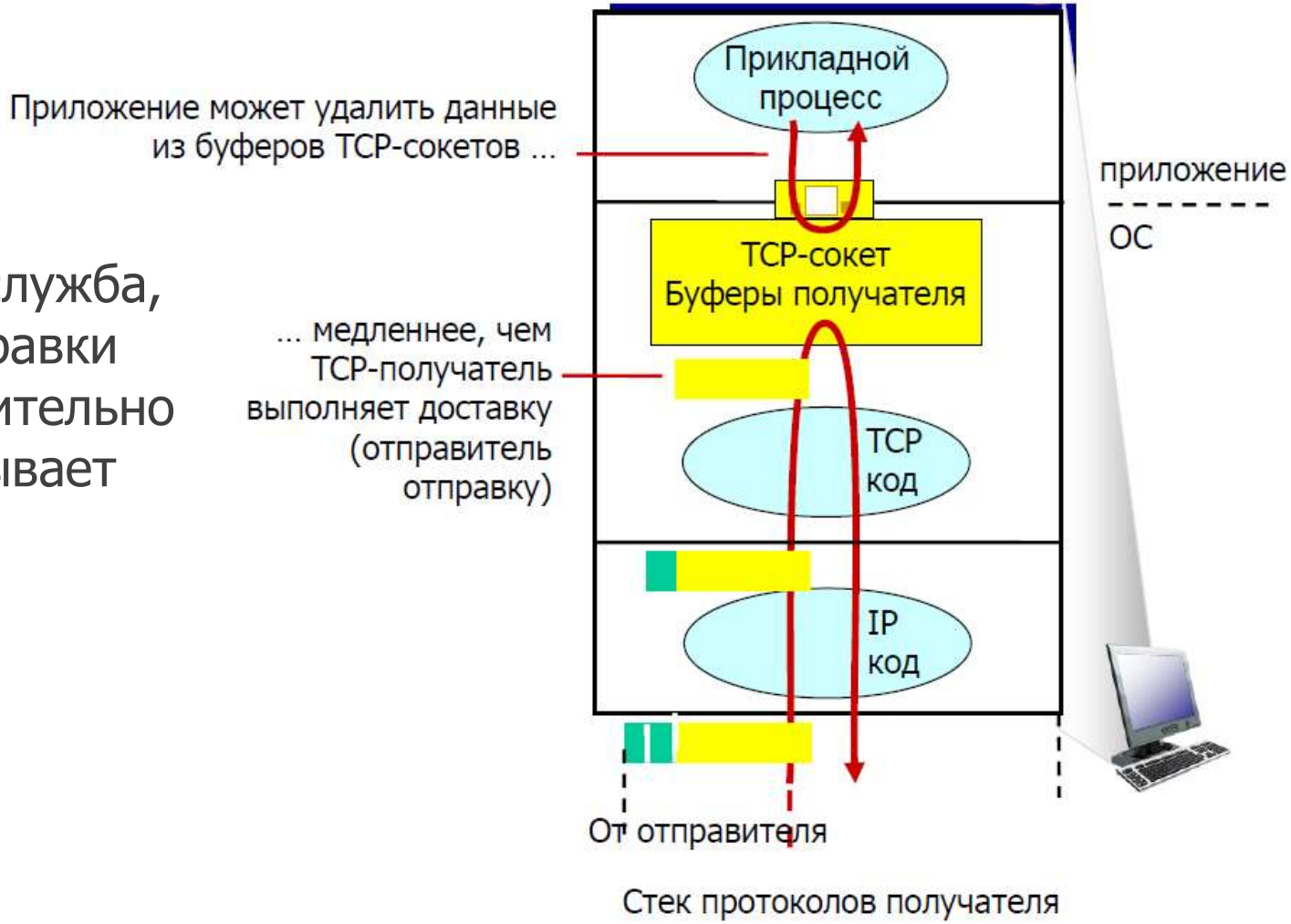
ТРАНСПОРТНЫЙ УРОВЕНЬ
Часть III

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Управление потоком TCP

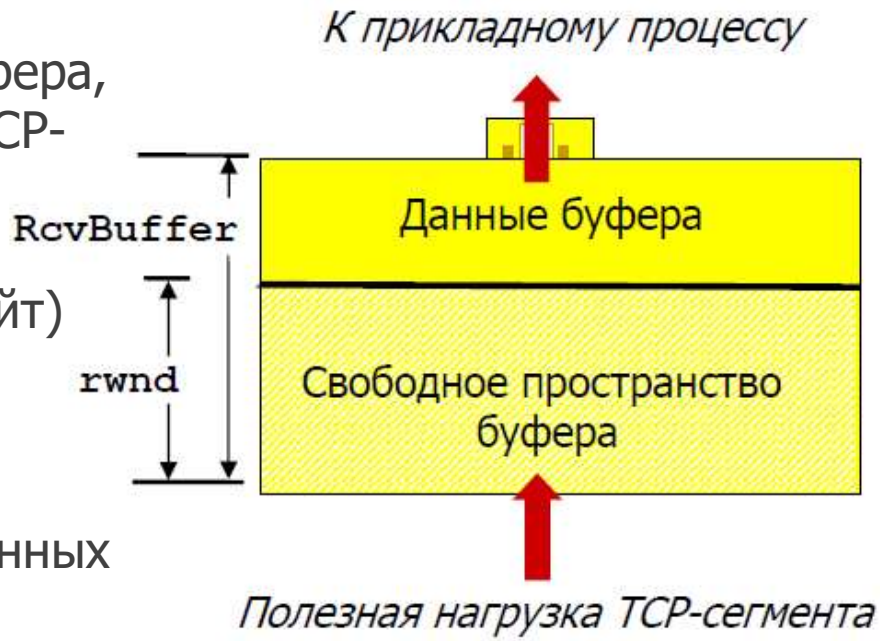
Управление потоком — это служба, регулирующая скорость отправки данных отправителем относительно скорости, с которой их считывает принимающее приложение



Управление потоком TCP

- Получатель «объявляет» свободное пространство буфера, включая значение **rwnd** (окно приема) в заголовок TCP-сегмента, направленного отправителю
  - Размер **RcvBuffer** устанавливается с помощью параметров сокета (по умолчанию обычно 4096 байт)
  - Многие операционные системы автоматически корректируют RcvBuffer

- Отправитель ограничивает количество неподтвержденных (передаваемых) данных значением получателя **rwnd**
- Осуществляется гарантированное отсутствие переполнения буфера



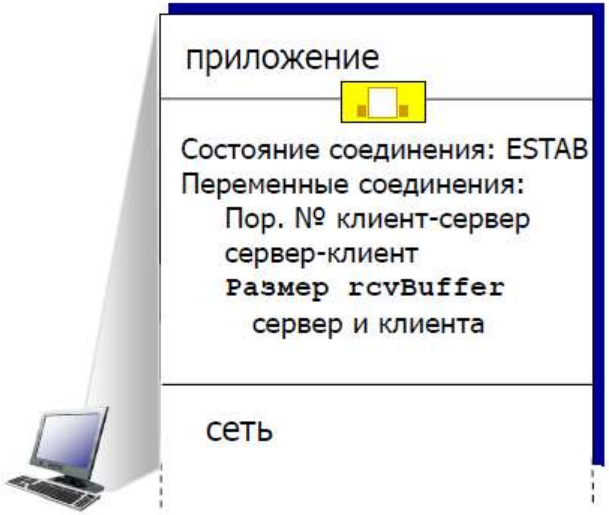
Буферизация на стороне получателя

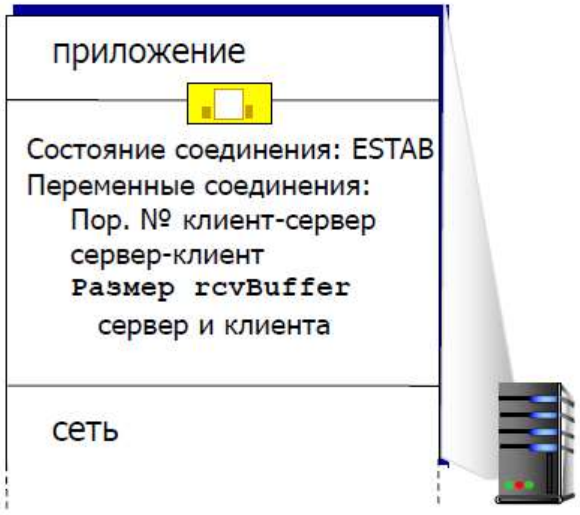
Управление соединением

Установление соединения перед обменом данными:

- Рукопожатие (каждая сторона знает, что другая хочет установить соединение)
- Согласование параметров соединения

Большинство наиболее распространенных сетевых атак, включая невероятно популярную потоковую атаку SYN, используют уязвимость в управлении TCP-соединением






```
Socket clientSocket =
  newSocket("имя_хоста", "номер_порта");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

Факультет математики и компьютерных наук

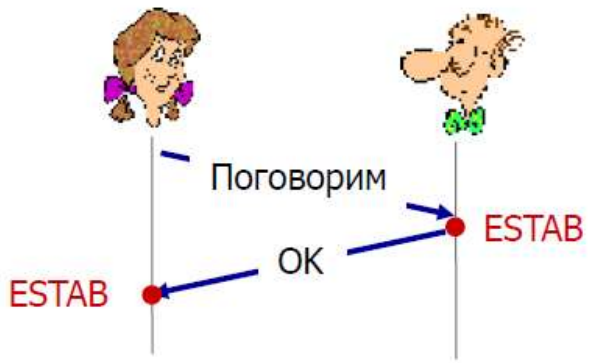
Согласование установления соединения

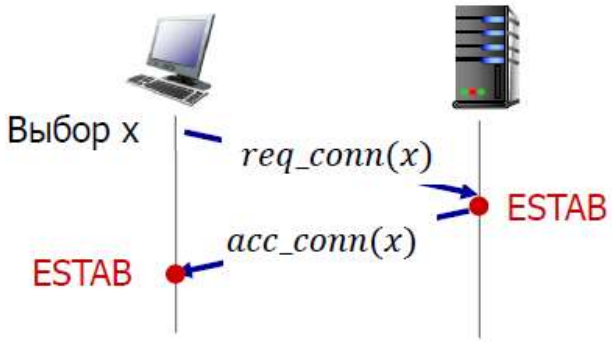
- Вопрос: всегда ли будет работать двух-шаговое рукопожатие в сети?

Возможные ситуации:

- Непостоянная задержка
- Повторно передаваемые сообщения (например, $req_conn(x)$) из-за потери сообщений
- Нарушение порядка сообщений
- Нет возможности «видеть» вторую сторону

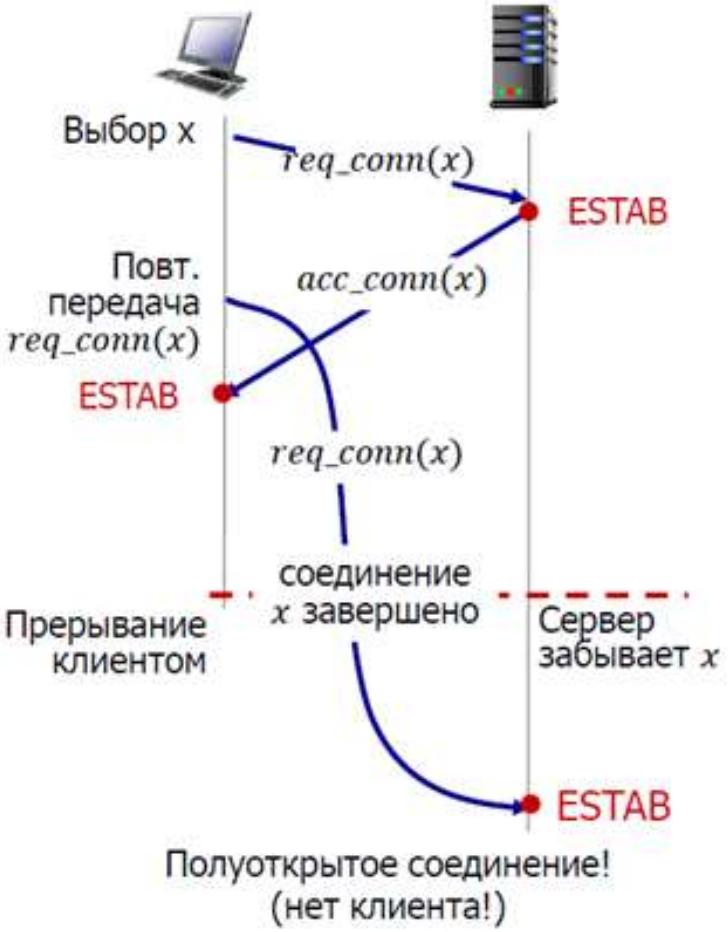
2-этапное рукопожатие:

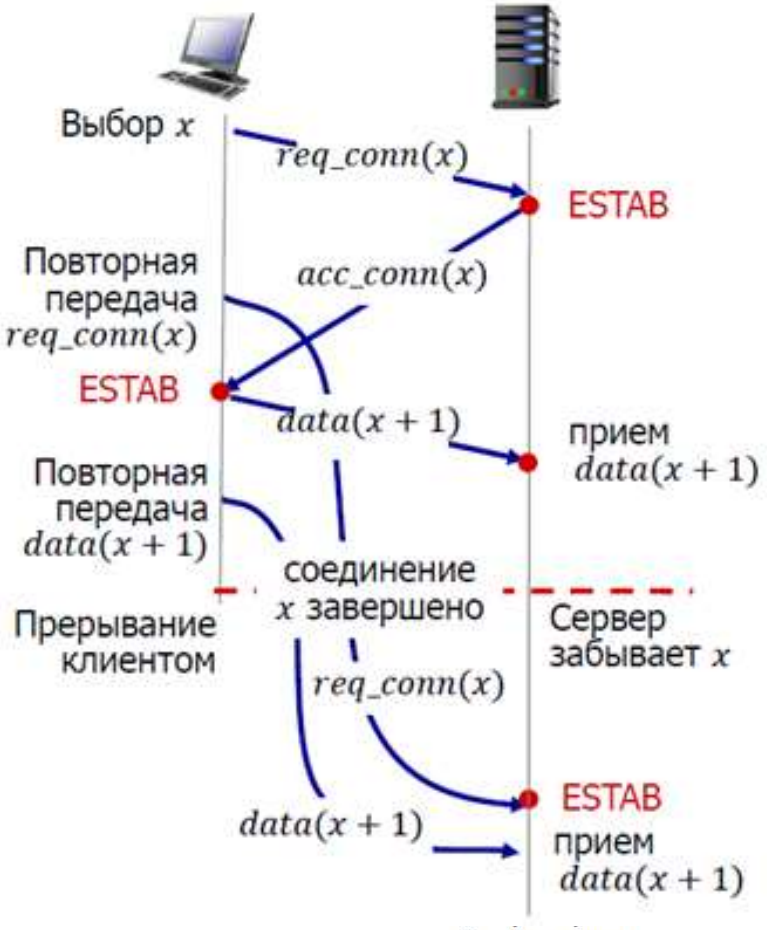




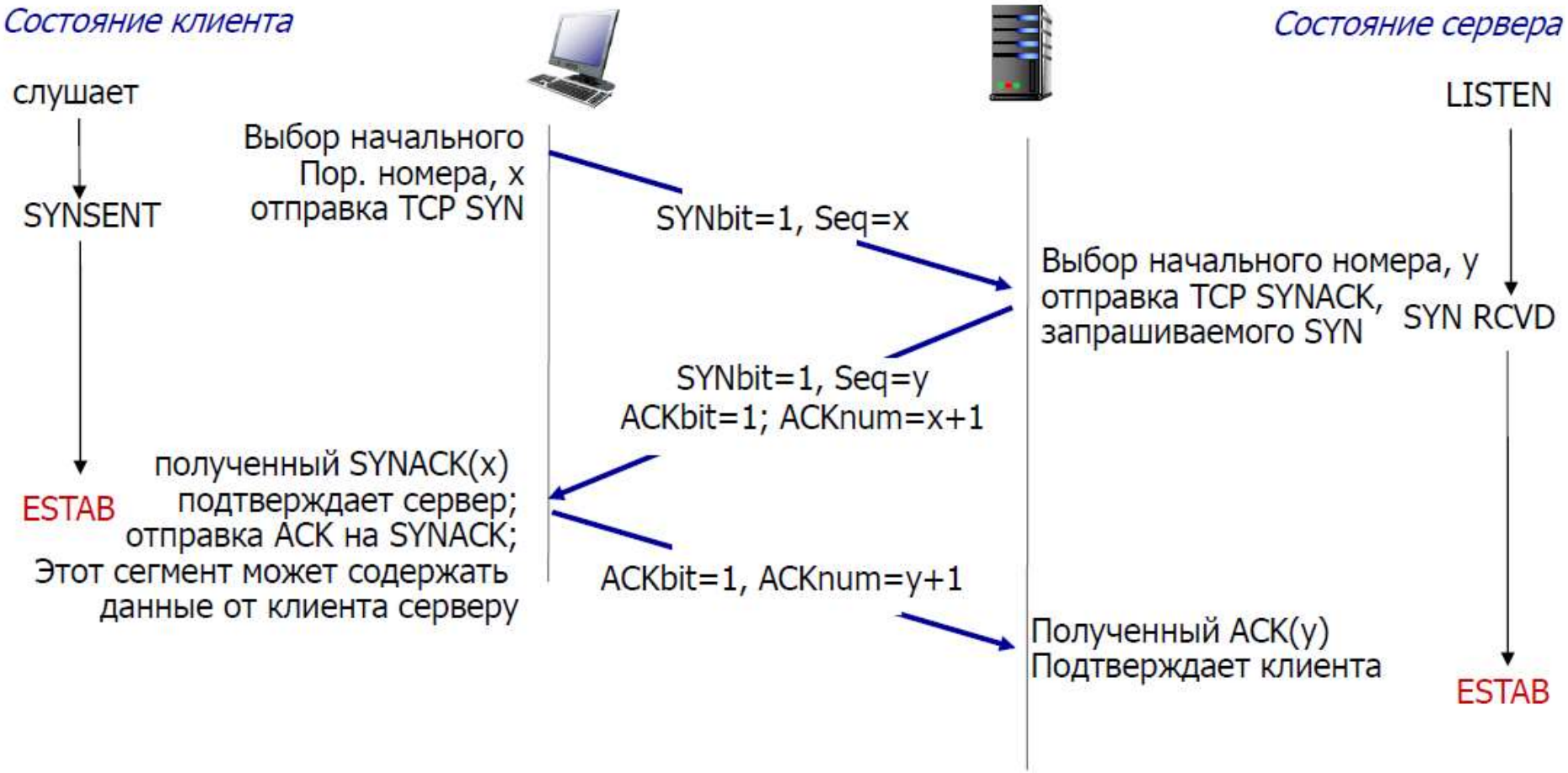
Согласование установления соединения

Двух-шаговое рукопожатие: сценарий отказа

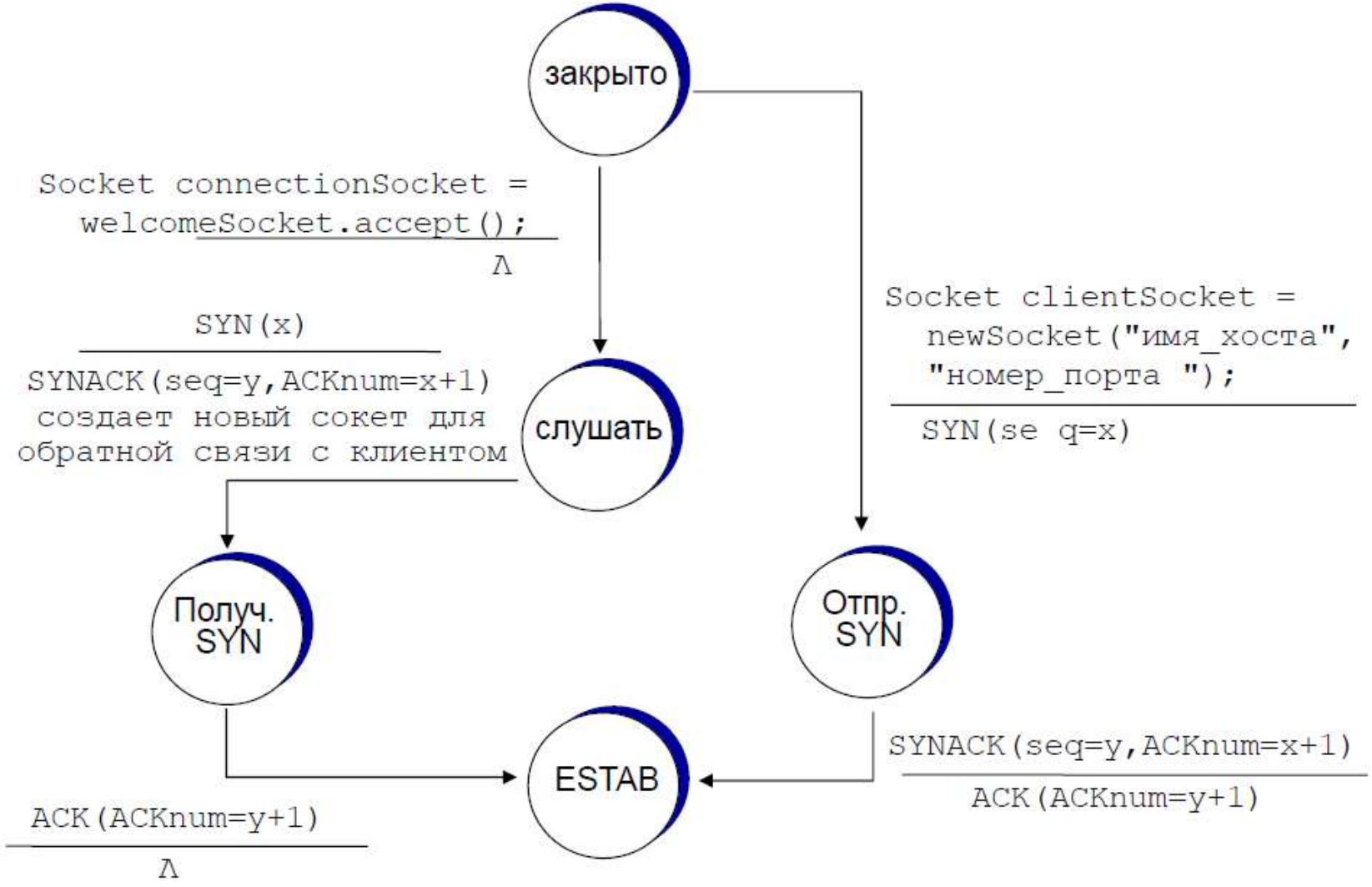




TCP: трех-шаговое рукопожатие



FSM-схема трех-шагового установления TCP-соединения



Факультет математики и компьютерных наук

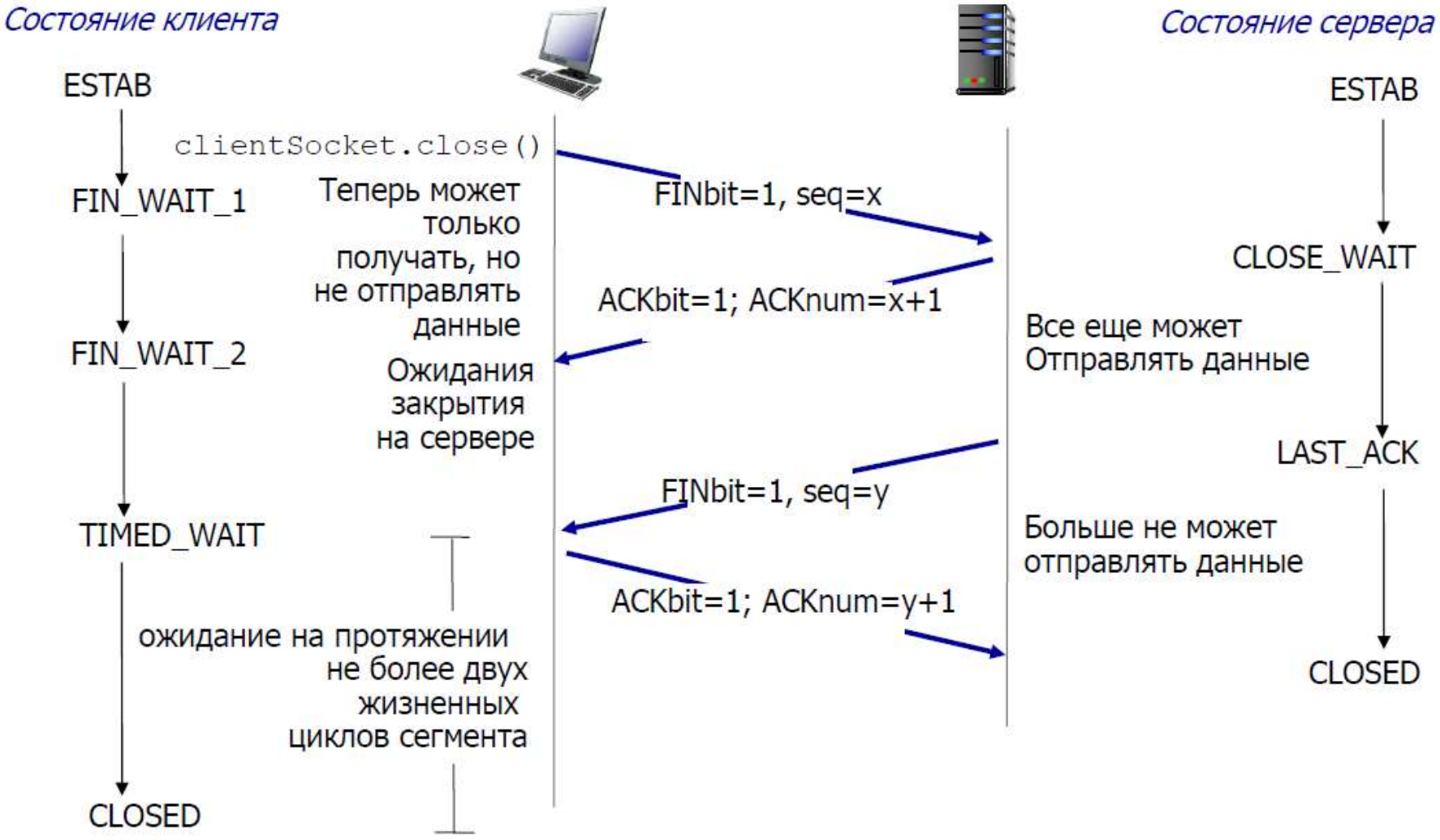
TCP: закрытие соединения

- Клиент и сервер закрывают соединение каждый со своей стороны
  - отправка TCP-сегмента с FIN битом равным 1

- Подтверждение получения FIN получением ACK-пакета
  - При получении FIN, ACK-пакет может быть объединен с собственным FIN

- Одновременный обмен FIN может быть обработан

TCP: закрытие соединения



Факультет математики и компьютерных наук

Атака SYN flood

Атакующий или атакующие отправляют большое количество TCP-сегментов SYN до того, как завершится третий этап установления соединения

При таком потоке SYN-сегментов ресурсы соединений сервера истощаются, поскольку они выделяются (но никогда не используются!) для наполовину открытых соединений

Это приводит к отказу в обслуживании настоящим клиентам

Как защититься?
SYN cookies. Сервер не выделяет ресурсы сразу, а отправляет хеш-код клиенту и ждет от него ответа. И не хранит cookies на своей стороне

Принципы управления перегрузками

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Принципы управления перегрузкой

Перегрузка:

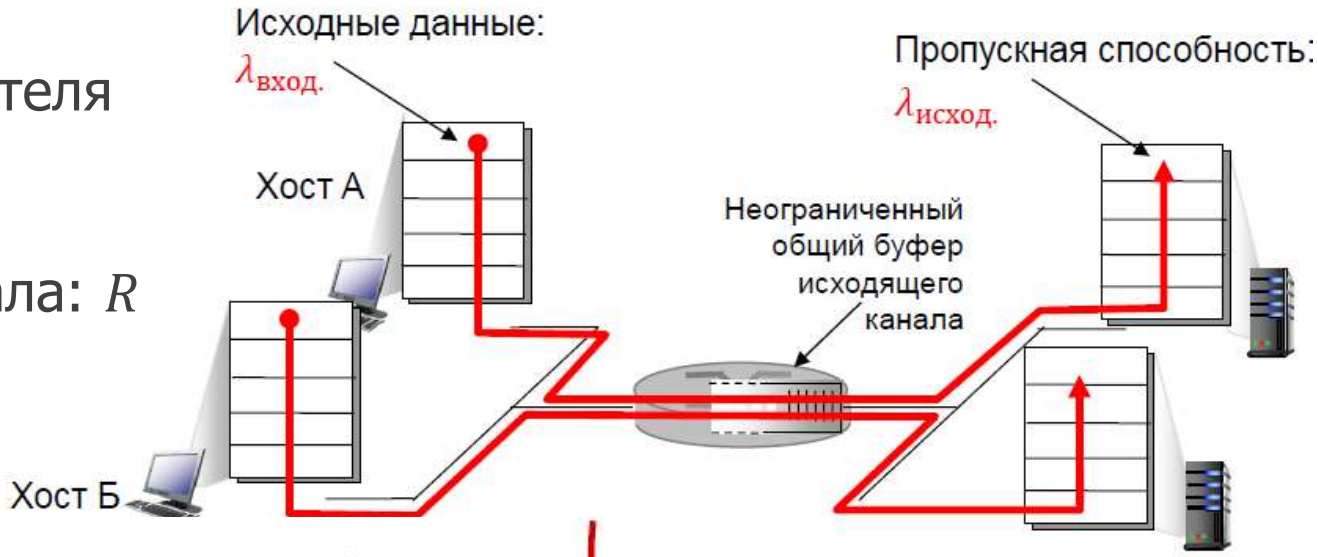
- Упрощенно: «слишком много источников слишком быстро отправляют слишком много данных для обработки в сети»

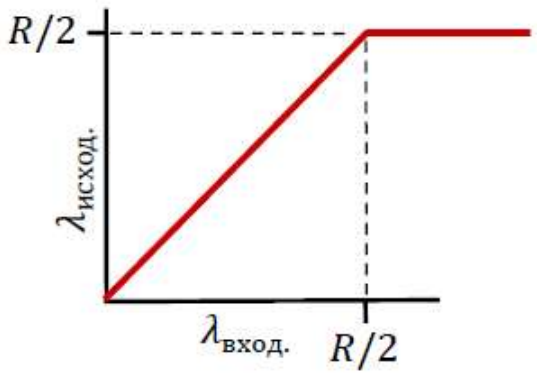
- Возможные варианты:
  - Потеря пакетов (переполнение буферов маршрутизаторов)
  - Длительные задержки (очереди в буферах маршрутизаторов)

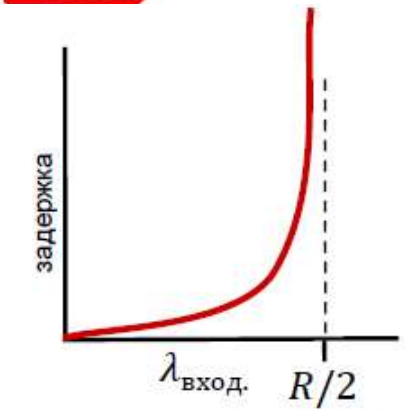
- Проблема из числа наиважнейших

Причины и следствия перегрузки: сценарий 1

- Два отправителя, два получателя
- Один маршрутизатор с неограниченным буфером
- Пропускная способность канала: R
- Нет повторных передач



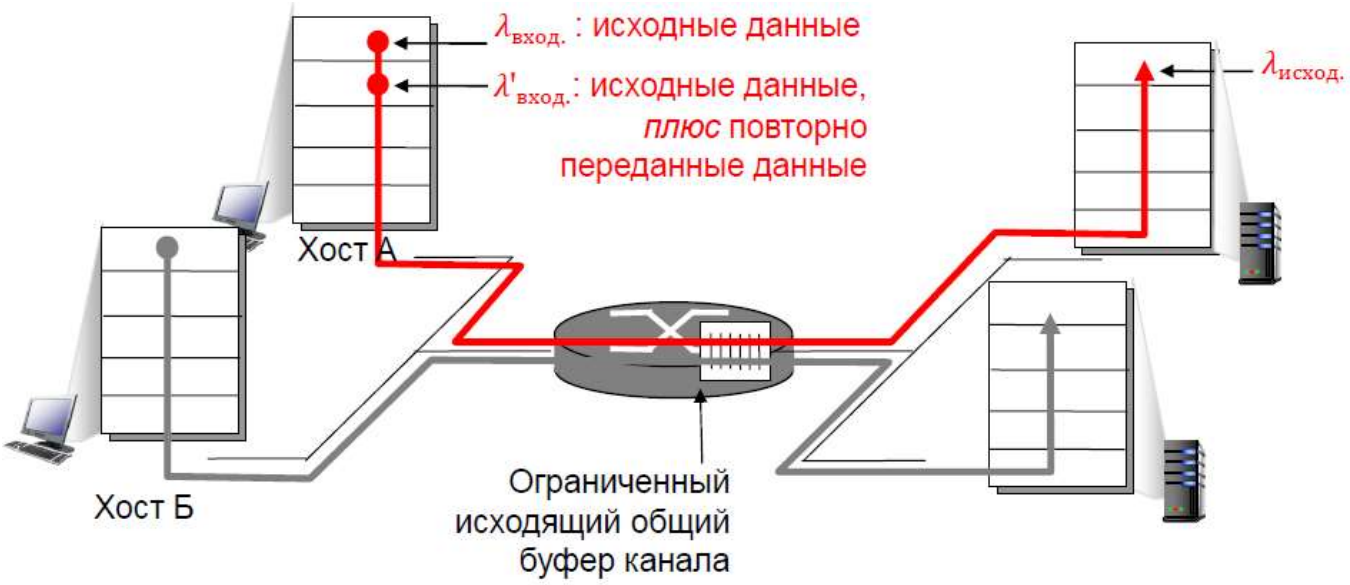




- Максимальная пропускная способность соединения: $R/2$
- Большие задержки прибытия сегментов, $\lambda_{вход.}$, приближения потенциала

Причины и следствия перегрузки: сценарий 2

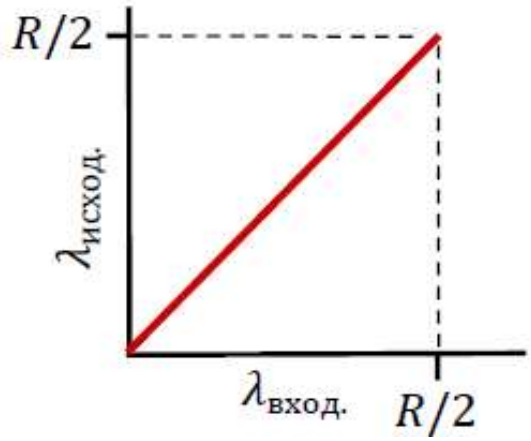
- Один маршрутизатор, ограниченные буферы
- Отправитель повторно передает пакеты, для которых истекло время
  - Вход уровня приложений = выход уровня приложений: $\lambda_{\text{вход}} = \lambda_{\text{исход}}$
  - Входные данные транспортного уровня включают повторные передачи: $\lambda'_{\text{вход.}} \geq \lambda_{\text{вход}}$

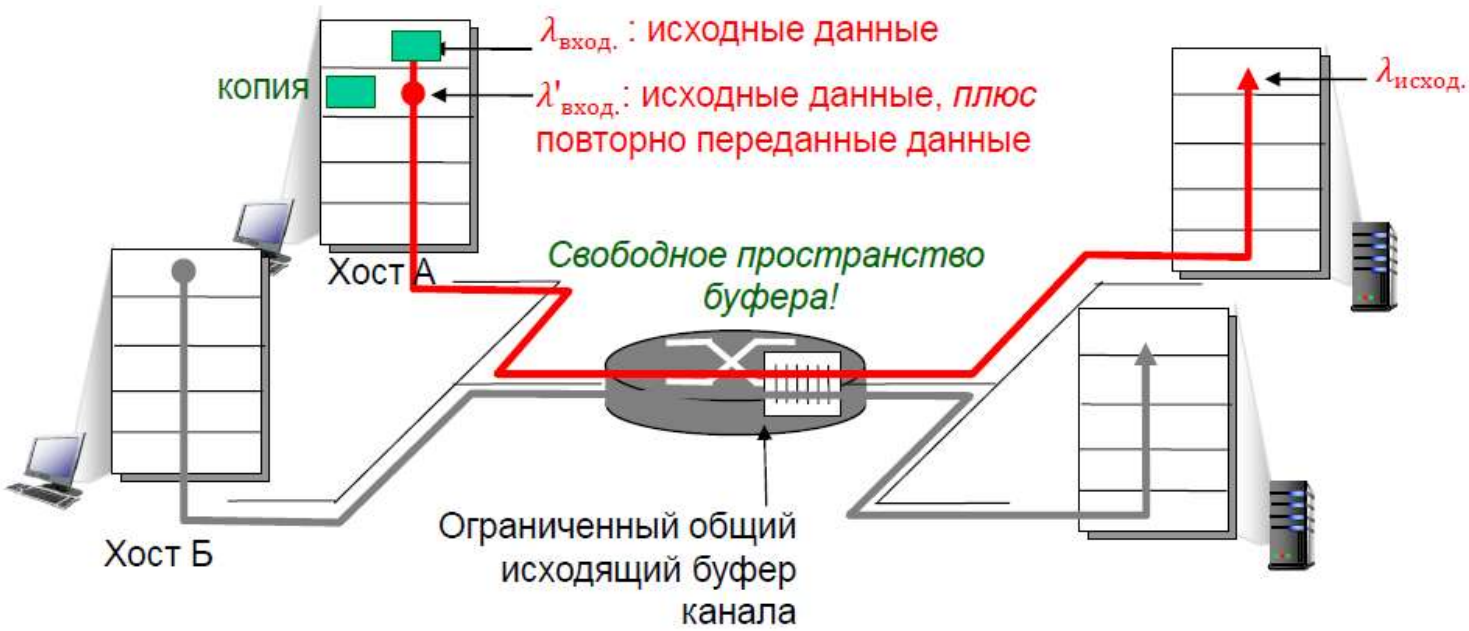


Причины и следствия перегрузки: сценарий 2

Идеализация: исчерпывающие сведения

Отправитель отправляет, только когда доступны буферы маршрутизатора (отправляющий хост выполняет передачу пакета, только когда буфер свободен. Но каким образом он об этом знает?)





Причины и следствия перегрузки: сценарий 2

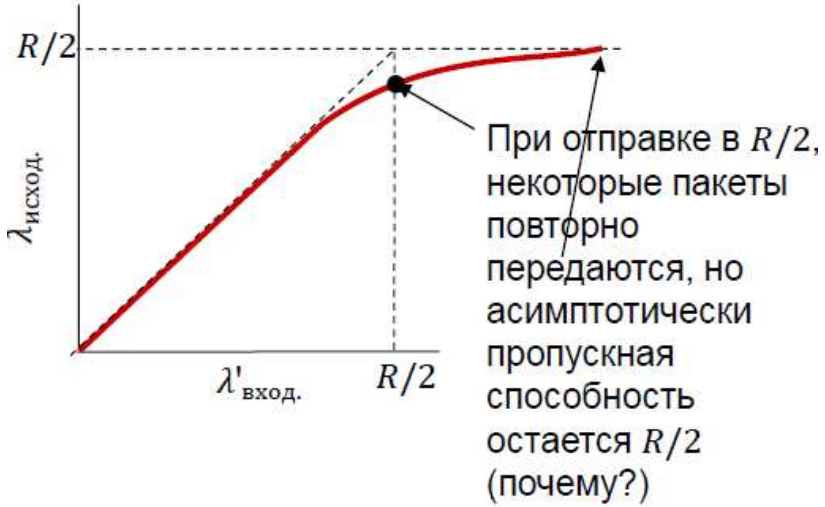
Идеализация: известны потери, пакеты могут быть потеряны, пропущены маршрутизатором из-за переполненного буфера

- Отправитель отправляет пакет повторно, только если знает, что он потерян

Причины и следствия перегрузки: сценарий 2

Идеализация: известны потери, пакеты могут быть потеряны, пропущены маршрутизатором из-за переполненного буфера

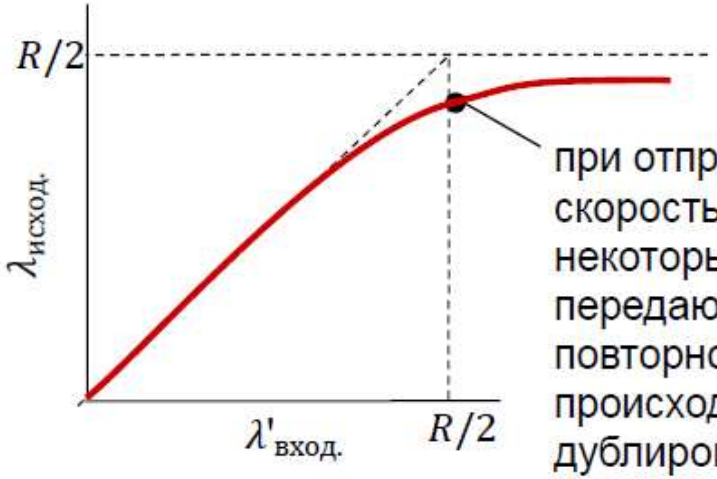
- Отправитель отправляет пакет повторно, только если знает, что он потерян



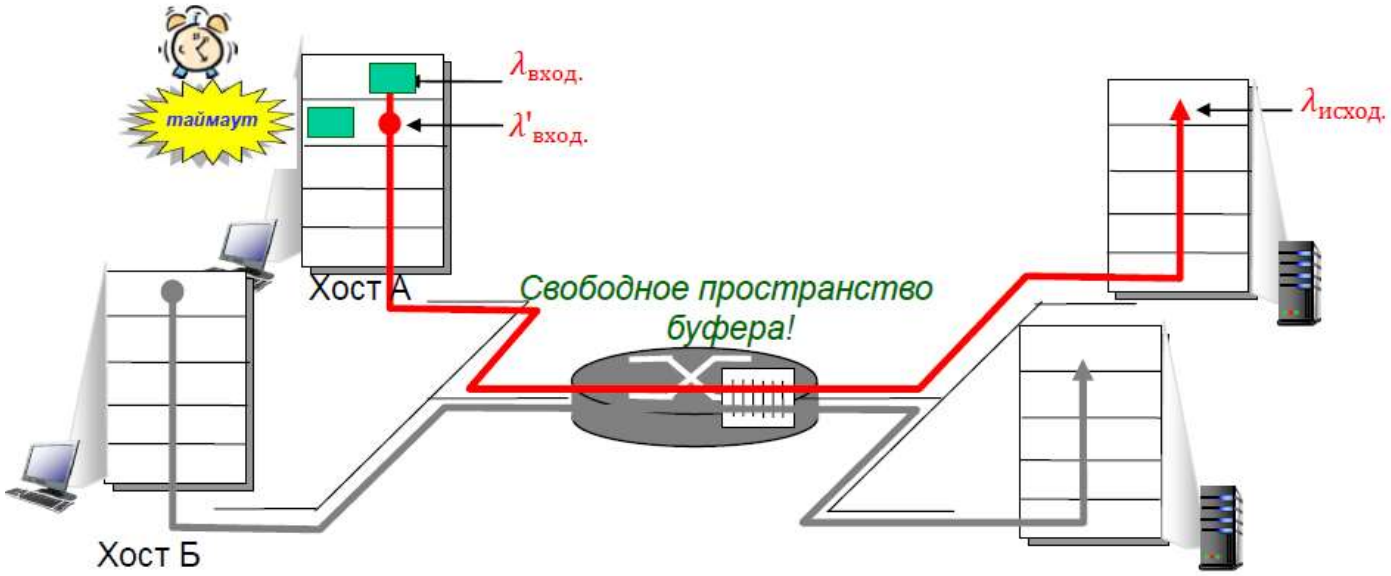
Причины и следствия перегрузки: сценарий 2

Реалистичная картина: дублирование

- Пакеты могут быть потеряны, пропущены маршрутизатором из-за переполнения буферов
- Преждевременный таймаут отправителя, отправка двух копий, каждая из которых доставляется



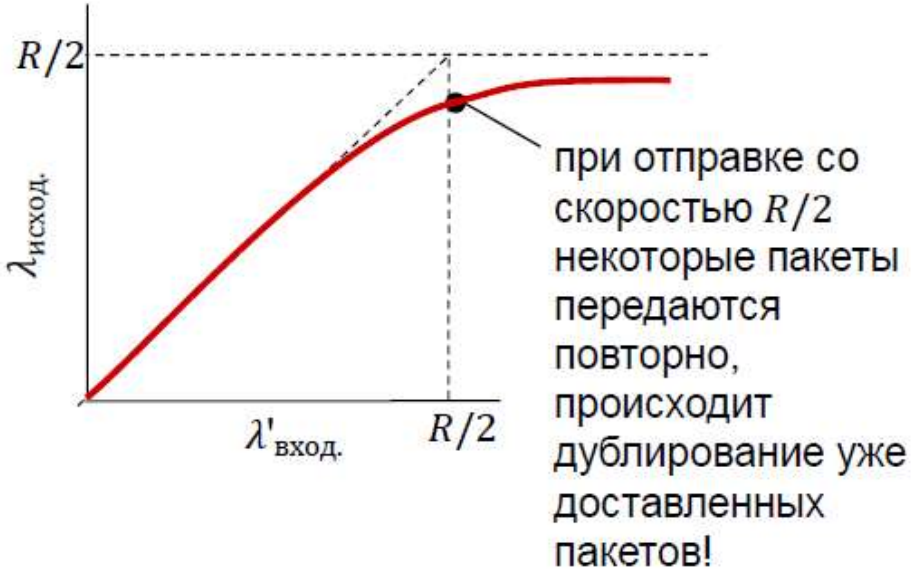
при отправке со скоростью $R/2$ некоторые пакеты передаются повторно, происходит дублирование уже доставленных пакетов!



Причины и следствия перегрузки: сценарий 2

Реалистичная картина: дублирование

- Пакеты могут быть потеряны, пропущены маршрутизатором из-за переполнения буферов
- Преждевременный таймаут отправителя, отправка двух копий, каждая из которых доставляется



«Следствия» перегрузки:

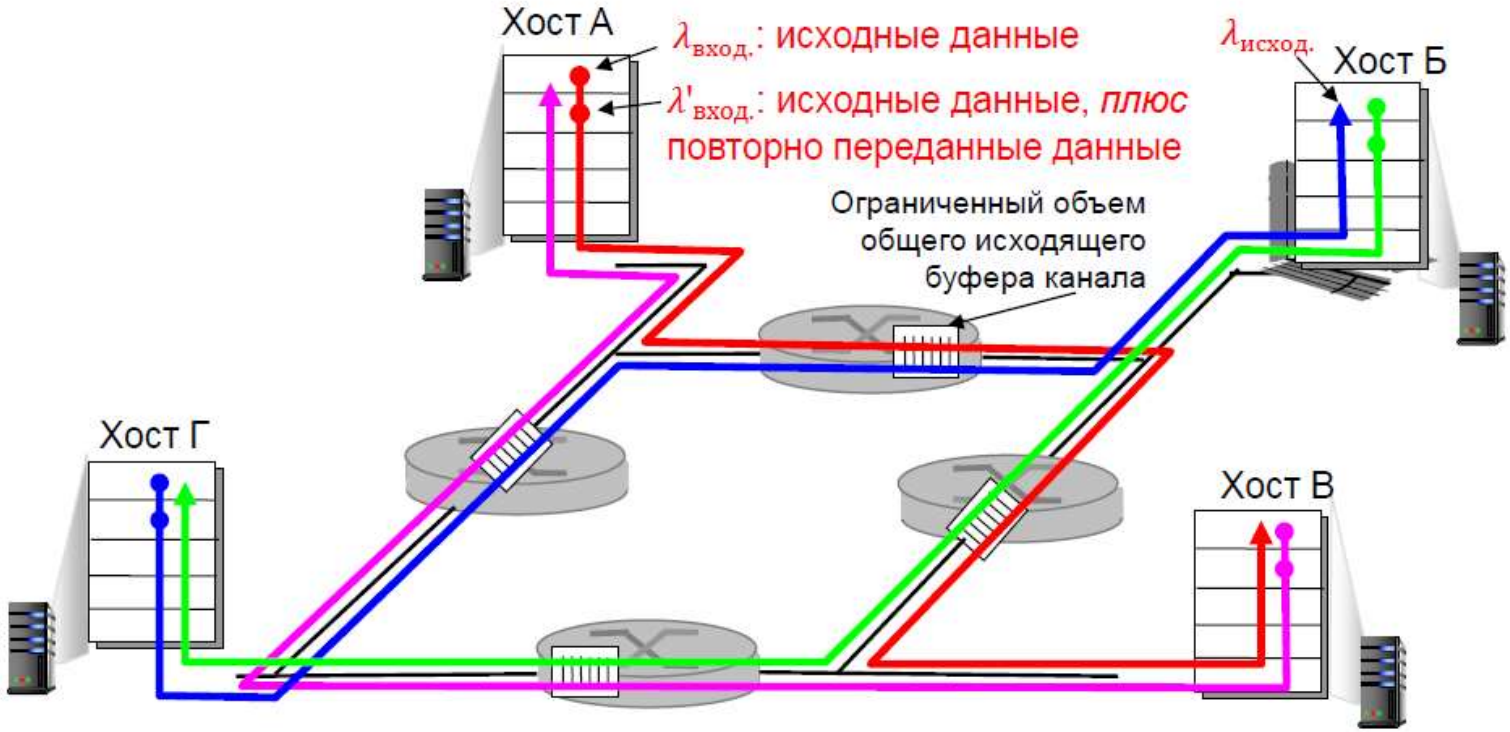
- Больше работы (повторные передачи)
- Ненужные повторные передачи: канал обрабатывает множественные копии пакетов => Снижение пропускной способности

Причины и следствия перегрузки: сценарий 3

- Четыре отправителя
- Путь в несколько переходов
- Таймаут/повторная передача

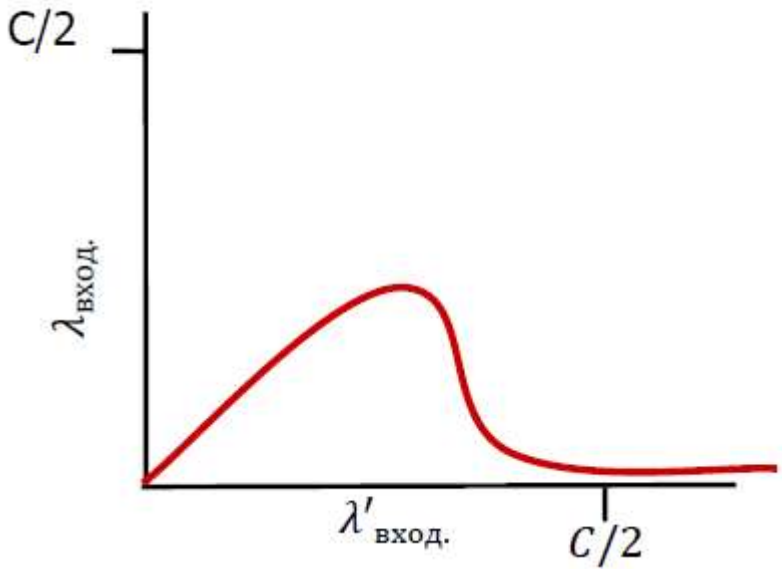
Вопрос: что произойдет при увеличении λвход. и λ'вход?

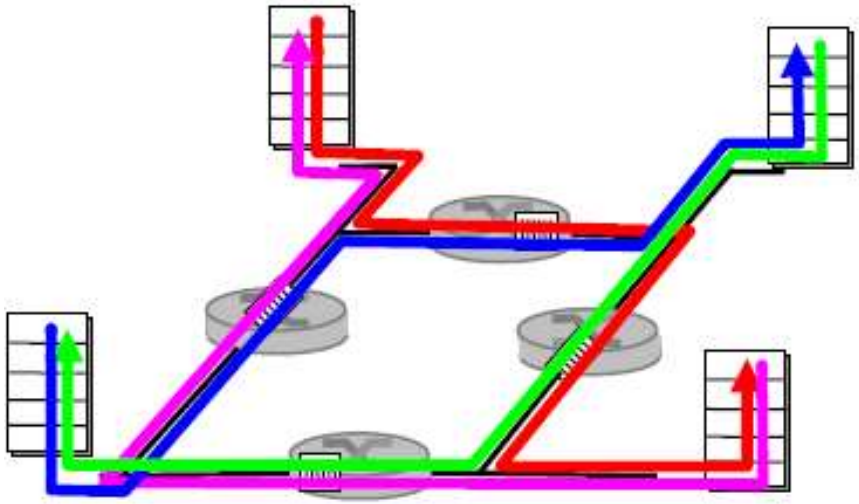
Ответ: при увеличении красного λ'вход. все поступающие синие пакеты могут быть пропущены, пропускная способность -> 0



Факультет математики и компьютерных наук

Причины и следствия перегрузки: сценарий 3





Другие «следствия» перегрузки:

- при отбрасывании пакета на одном из маршрутизаторов работа каждого предыдущего маршрутизатора по передаче пакета получателю считается проделанной впустую!

Подходы к управлению перегрузкой

Два основных подхода к управлению перегрузкой:

- Управление перегрузкой на конечных системах:
  - Нет явного отклика от сети
  - На перегрузку указывают потери и задержки на конечных системах
  - Используется в протоколе TCP

- Сетевое управление перегрузкой:
  - Маршрутизаторы дают отклик конечным системам о нагрузках в сети
    - Одноразрядный индикатор перегрузки (SNA, DECbit, TCP/IP ECN, ATM)
    - Явное указание скорости отправителю

Пример управления перегрузкой ATM ABR

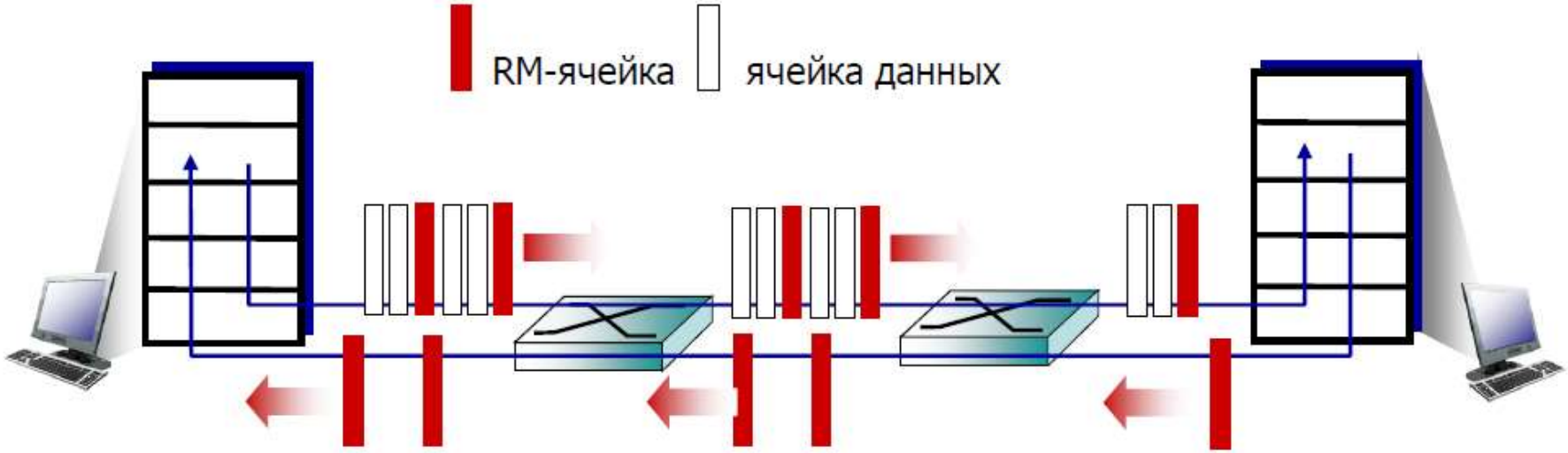
ABR: доступная скорость:

- «эластичный сервис»
- если путь отправителя «недогружен»:
  - Отправитель должен использовать доступную пропускную способность канала
- Если путь отправителя перегружен:
  - Отправитель замедляет скорость до минимальной гарантированной скорости

RM-ячейки (управление ресурсами):

- Передаваемые отправителем, перемежаются с ячейками данных
- Биты в RM-ячейке устанавливаются коммутаторами («поддержка со стороны сети»)
  - NI бит: нет увеличения скорости (средняя перегрузка)
  - CI бит: индикатор перегрузки
- RM-ячейки возвращаются отправителю от получателя

Пример управления перегрузкой ATM ABR



- Двух байтовое поле ER (explicit rate, явная скорость) в RM-ячейке
  - Перегруженный коммутатор может уменьшить значение ER в ячейке
  - Скорость ухода данных от отправителя – максимальная поддерживаемая скорость на пути

- EFCI бит в ячейках данных: получает значение 1 в перегруженном коммутаторе
  - Если ячейка данных, предшествующая RM-ячейке, имеет установленный бит EFCI, получатель устанавливает CI-бит в возвращаемой RM-ячейке

Механизм управления перегрузками протокола TCP

Санкт-Петербургский государственный университет

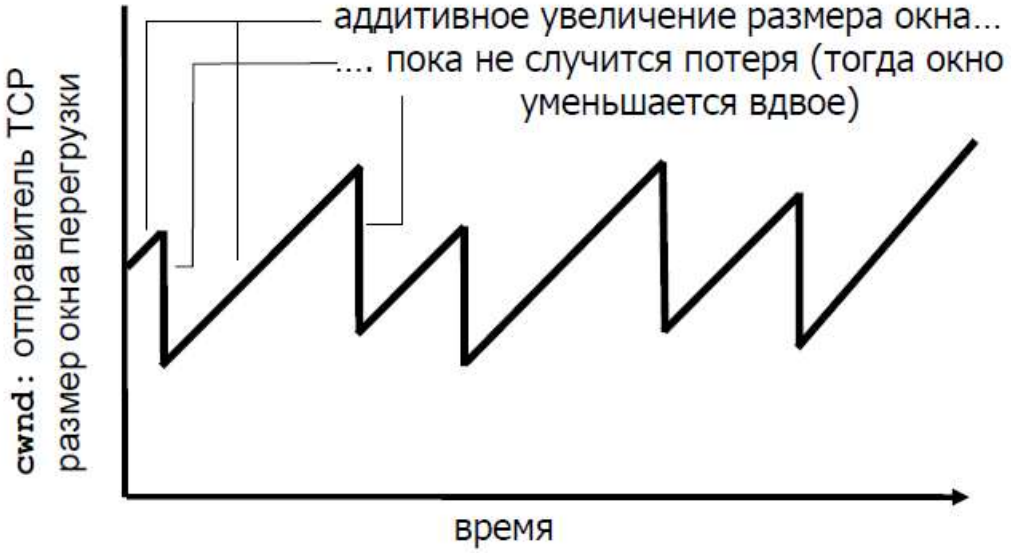
Факультет математики и компьютерных наук

Управление перегрузкой TCP: аддитивное ускорение и мультипликативное замедление

Суть: отправитель увеличивает скорость передачи (размер окна), зондируя доступную ширину канала, пока не случится потеря

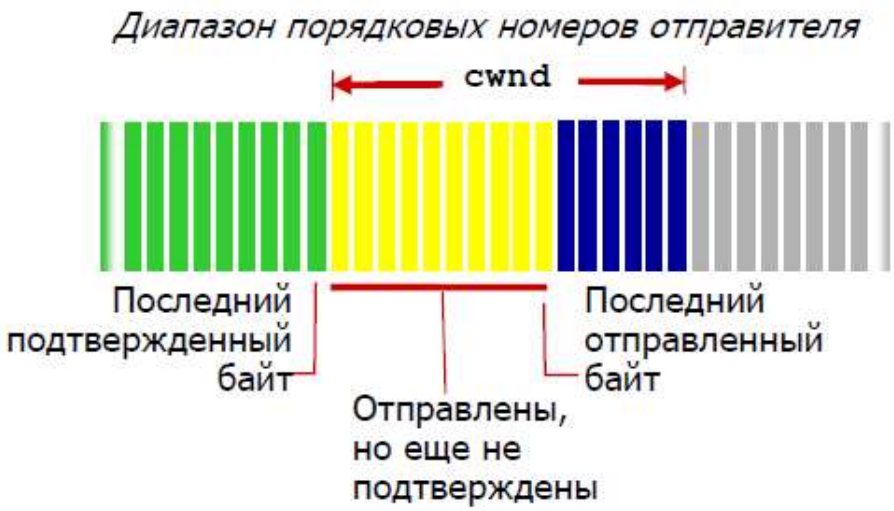
- Аддитивное ускорение: увеличение cwnd на 1 MSS (maximum segment size) каждые RTT до обнаружения потери
- Мультипликативное замедление: cwnd уменьшается в половину после потери

Алгоритм AIMD демонстрирует зубчатое поведение: зондирование ширины канала



Факультет математики и компьютерных наук

Управление перегрузкой TCP



- Предел передачи отправителя:

$$\text{LastByteSent-LastByteAcked} \le \text{cwnd}$$

- **cwnd** – это динамическая функция от перегрузки сети

Скорость отправки TCP:

- приблизительно: отправить **cwnd** байт, ждать RTT до получения всех ACK-пакетов, затем отправить еще

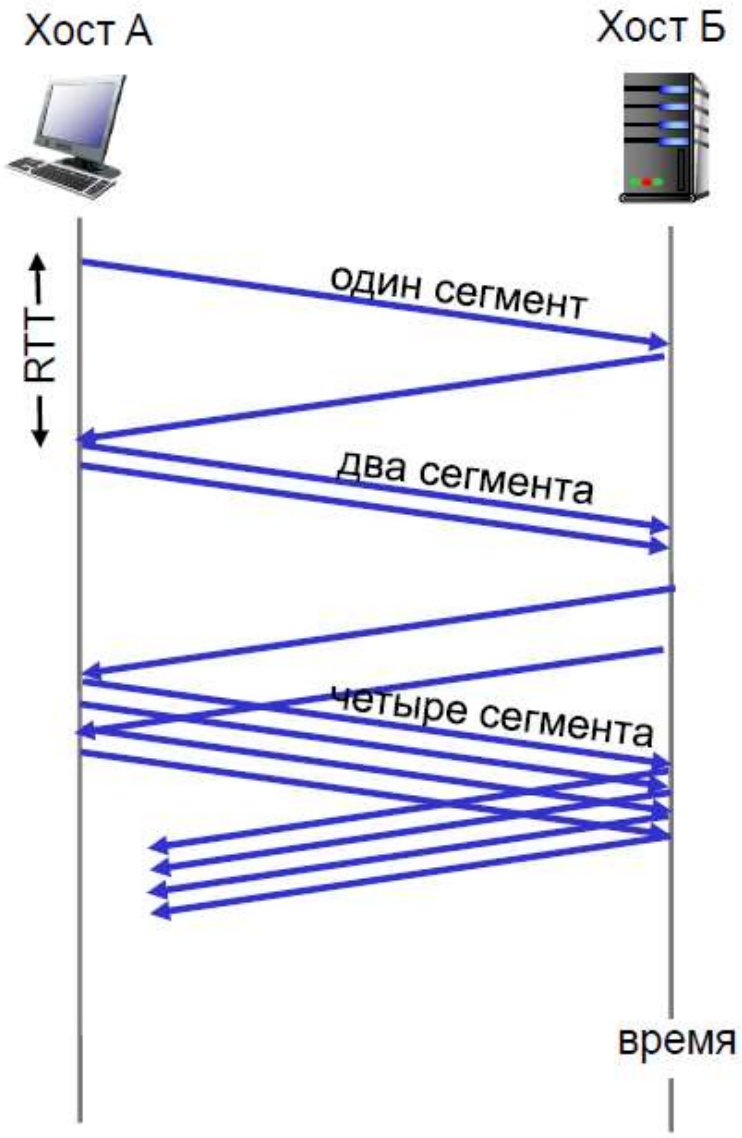
$$\text{скорость} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ байт/сек}$$

Регулируя значение **cwnd**, отправитель может управлять скоростью передачи данных соединения

TCP: Медленный старт

- Скорость увеличивается экспоненциально с начала соединения и до первого события потери:
  - начальное cwnd = 1 MSS
  - удвоение cwnd каждые RTT
  - увеличение cwnd при получении каждого ACK

- Резюме: начальная скорость невысока, но возрастает экспоненциально



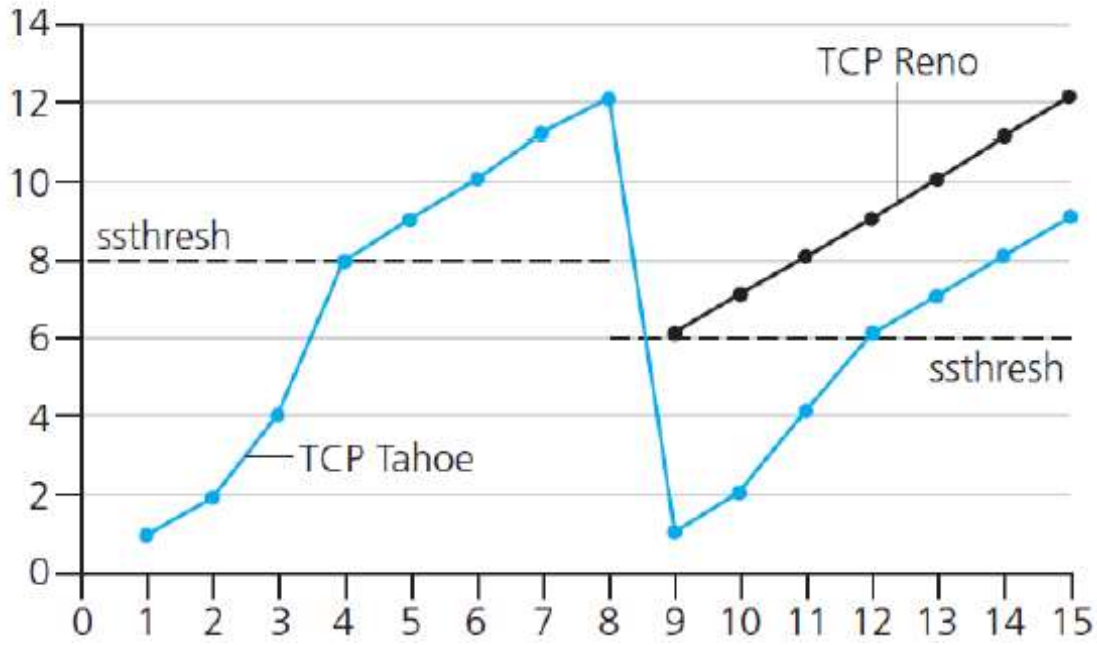
TCP: обнаружение и обработка потерь

- На потери указывает таймаут:
  - cwnd становится равным 1 MSS
  - затем окно экспоненциально увеличивается (как при медленном старте) до порогового значения, после чего увеличивается линейно

- На потери указывают 3 дублирующих ACK-пакета:
  - дублирующие ACK-пакеты указывают на необходимость ускоренной повторной передачи данных по сети
  - TCP RENO:
    - cwnd уменьшается вдвое, затем окно линейно увеличивается
  - TCP Tahoe:
    - всегда устанавливает cwnd равным 1 (и при таймауте, и при 3 дублирующих ACK-пакетах)

TCP: переход из медленного старта

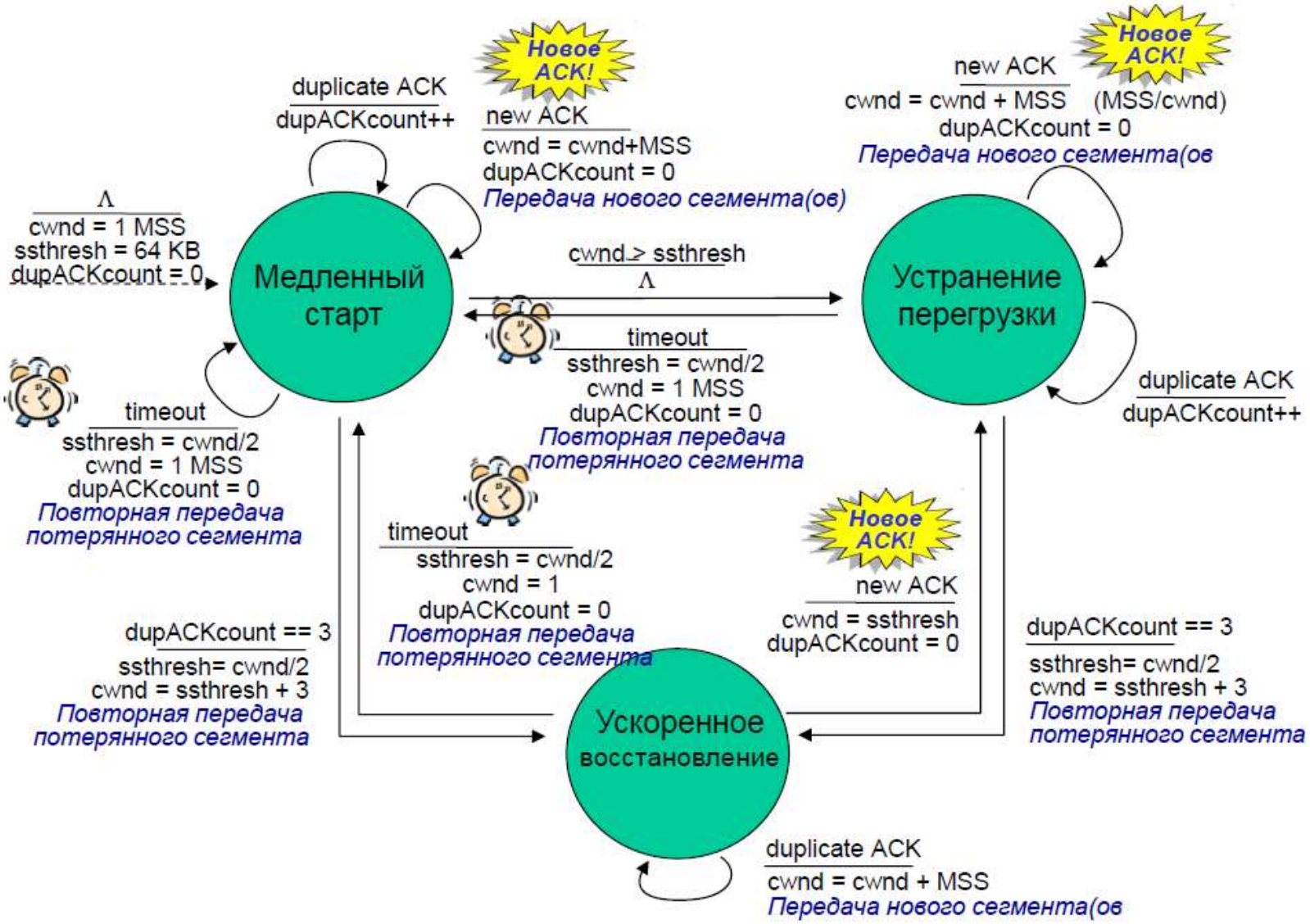
- ***Вопрос***: когда экспоненциальное ускорение должно смениться линейным?
- ***Ответ***: когда *cwnd* будет равно ½ от собственного значения до таймаута



Реализация:

- Изменяемое значение **ssthresh**
- В случае потери, ssthresh будет равно 1/2 от последнего значения *cwnd* (до события потери)

Резюме: управление перегрузкой TCP Reno

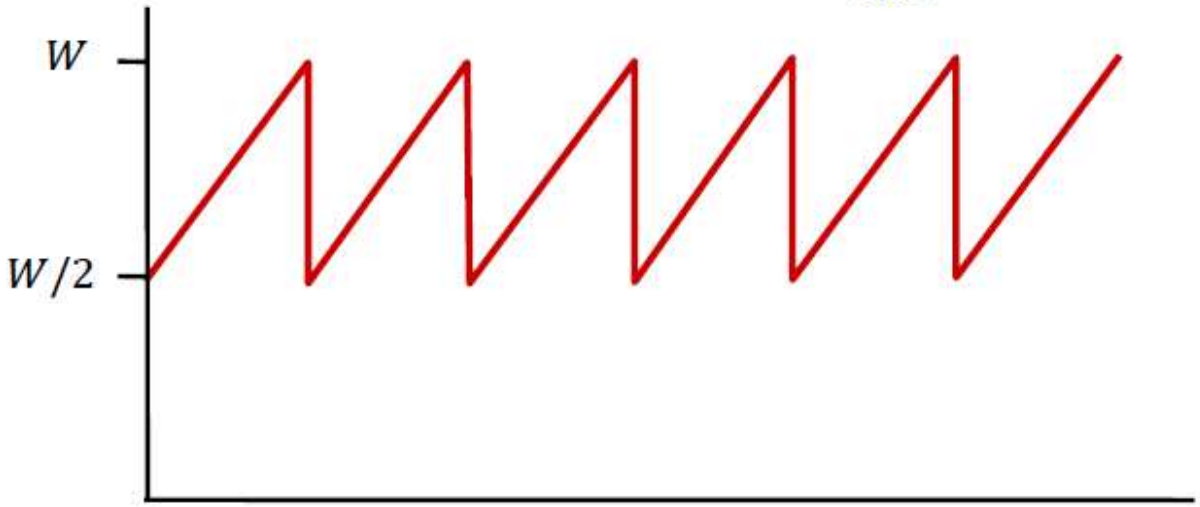


Факультет математики и компьютерных наук

Пропускная способность TCP

- Средняя пропускная способность TCP-соединения - это функция от размера окна RTT?
  - Игнорируем медленный старт, допускаем, что всегда есть данные для отправки
- W: размер окна (в байтах) при потерях
  - Средний размер окна (количество передаваемых байт) равно ¾W
  - Средняя пропускная способность равна 3/4W в RTT

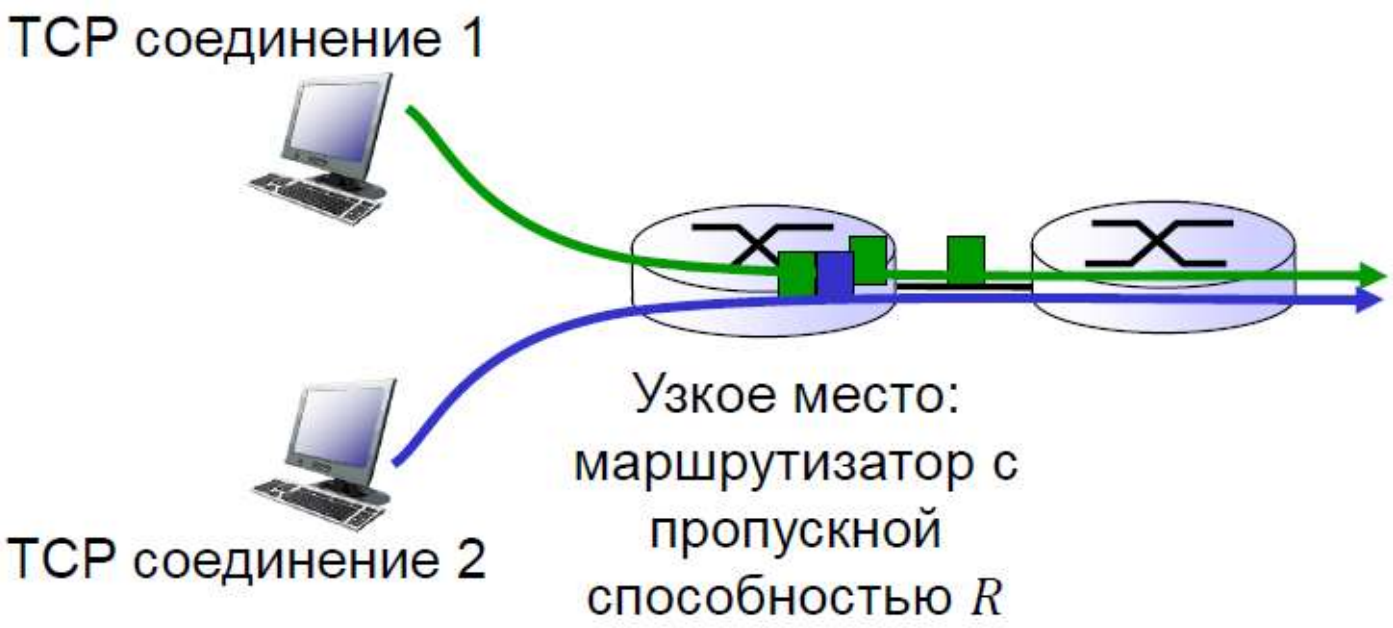
Ср. пропускная способность TCP $= \frac{3}{4}\frac{W}{RTT}$ Байт/сек



Факультет математики и компьютерных наук

TCP: выравнивание скорости передачи

Цель выравнивания: если K TCP-сеансов совместно используют узкий канал ширины R, каждое соединение должно получить среднюю скорость R/K

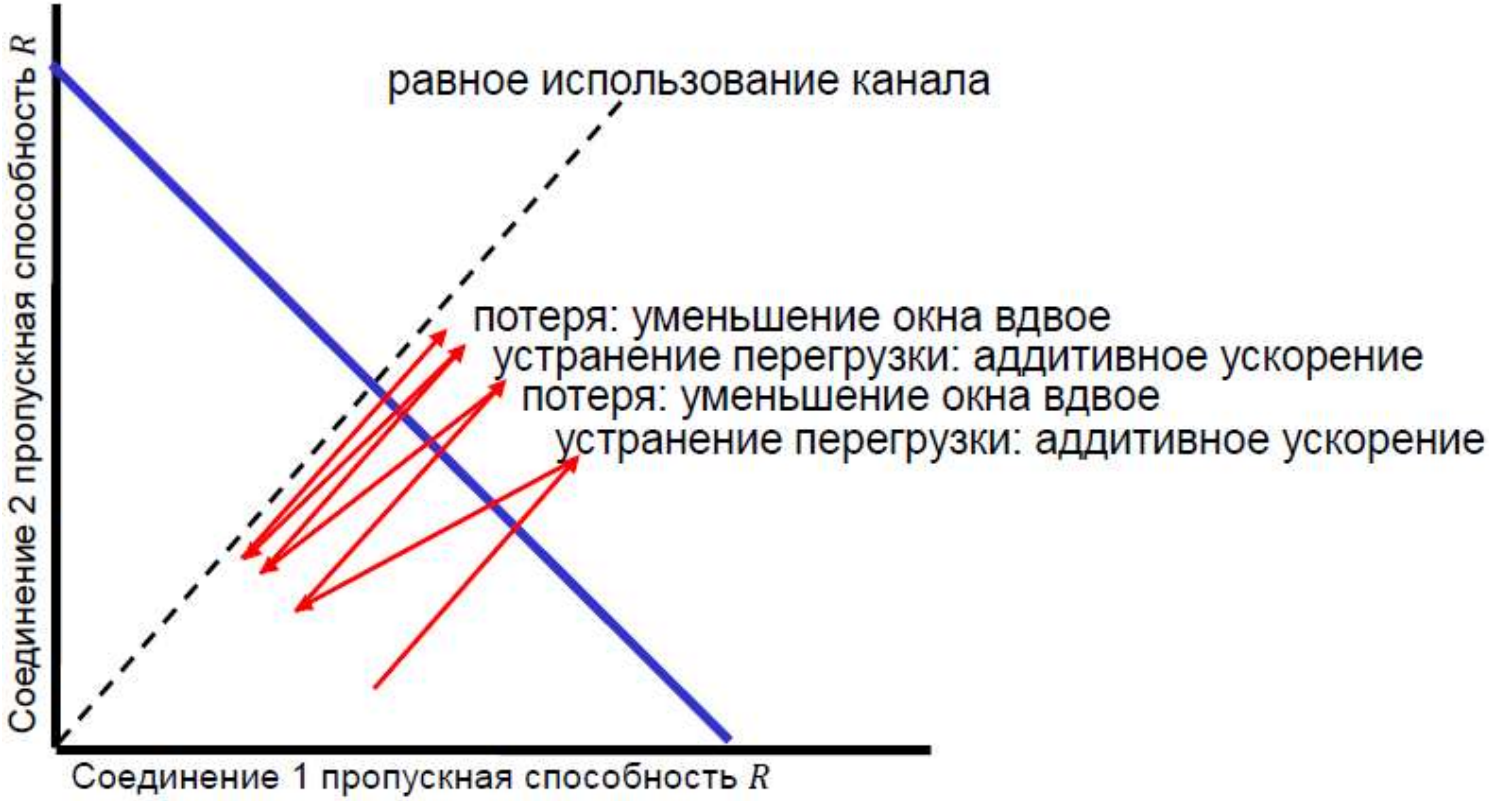


Так как TCP-соединения могут устанавливаться в разные моменты времени и тем самым иметь неравные размеры окон перегрузки, возникает вопрос: обеспечивает ли выравнивание скоростей алгоритм аддитивного увеличения и мультипликативного уменьшения протокола TCP?

Почему в TCP выполняется выравнивание скорости передачи?

Два параллельных сеанса:

- Аддитивное ускорение дает уклон 1, при увеличении пропускной способности
- Мультипликативное замедление пропорционально снижению пропускной способности



Выравнивание скорости передачи

Выравнивание скорости и UDP

- Мультимедийные приложения зачастую не используют протокол TCP по причине нежелательного снижения скорости из-за управления перегрузкой
- Напротив, используют UDP:
  - Отправляют аудио/видео с постоянной скоростью, не чувствительны к потере пакетов

Выравнивание скорости параллельных TCP соединений

- Приложение может открывать несколько одновременных соединений между двумя хостами
- Так поступают веб-браузеры
- Например, скорость канала R для 9 действующих соединений:
  - Новое приложение, запрашивающее 1 TCP соединение, получает скорость $R/10$
  - Другое новое приложение, запрашивающее 11 TCP соединений, получит $R/2$ скорости

Будущее TCP

- Пример: сегменты по 1500 байт, 100мс RTT, необходима пропускная способность 10 Гбит/с
- Требуется W = 83 333 передаваемых сегментов
- Выраженная в вероятных потерях сегментов L пропускная способность [Mathis 1997]:

$$\text{Пропускная способность TCP} = \frac{1{,}22 \times MSS}{RTT\sqrt{L}}$$

Для получения пропускной способности 10 Гбит/сек, частота потерь должна быть $L = 2 \times 10^{-10}$

- Разрабатываются новые версии протокола TCP для высоких скоростей

Заключение

- Принципы служб транспортного уровня:
  - Мультиплексирование, демультиплексирование
  - Надежная передача данных
  - Управление потоком
  - Управление перегрузкой
  - Выравнивание скорости передачи

- Реализация в сети Интернет:
  - UDP
  - TCP

Далее:

- Покидаем «периферию» сети (прикладной и транспортный уровни)
- Продвигаемся в «ядро» сети